Optimización en Ciencia de Datos

Clase 3: redes neuronales y optimización no convexa

Pablo A. Lotito Victoria Orlando

Modelización Matemática
Facultad de Ciencias Exactas – UNCPBA

2.º cuatrimestre 2026









Objetivos

Aprendizaje supervisado y Redes neuronales artificiales

- Formular redes neuronales como clases de funciones paramétricas
- Analizar el problema de optimización no convexa de entrenamiento
- Comprender la estructura de las funciones de pérdida
- Estudiar arquitecturas estructuradas (CNN, GNN)

Puente con lo que ya vimos en Modelización

Lo que ya vimos	**Unidad**	**Qué pasa con las redes**
Reglas de composición que preservan convexidad	U2–U3	Dejan de valer: la red no es convexa en θ
Condiciones de primer y segundo orden	U3–U4	Siguen valiendo, pero sólo localmente
Local = global en problemas convexos	U4	Se pierde: aparecen puntos silla y mesetas
Problema de mínima norma ($\min \|x\|$, $Ax = b$)	U9	Explica la regularización implícita
Ridge y pérdida logística (clases 1 y 2)	—	Son *weight decay* y la entropía cruzada

La pregunta de la clase

¿Qué garantías del caso convexo sobreviven cuando entrenamos una red neuronal, y cuáles no?

Modelización

Aprendizaje supervisado

Problema: dados n pares $\{(x_i, y_i)\}_{i=1}^{n}$ con $x_i, y_i \in \mathbb{R}$, hallar f tal que $f(x_i) \approx y_i$.

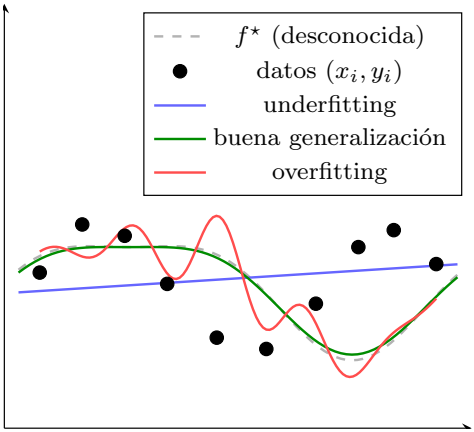
Diyuntiva

- **Precisión** (en train): f pasa cerca de todos los puntos conocidos.
- **Generalización** (en test): f predice bien en puntos *nuevos*.

Dos extremos malos:

- **Underfitting:** f demasiado simple, no captura el patrón.
- **Overfitting:** f memoriza el ruido, no generaliza.

El objetivo del aprendizaje **no** es minimizar el error en los datos observados, sino estimar bien la función subyacente.



Clases de funciones - parámetros

Idea: Buscar f en una **clase paramétrica** $\mathcal{F} = \{f_\theta : \theta \in \mathbb{R}^p\}$ y buscamos el mejor θ.

Ejemplo: polinomios de grado d

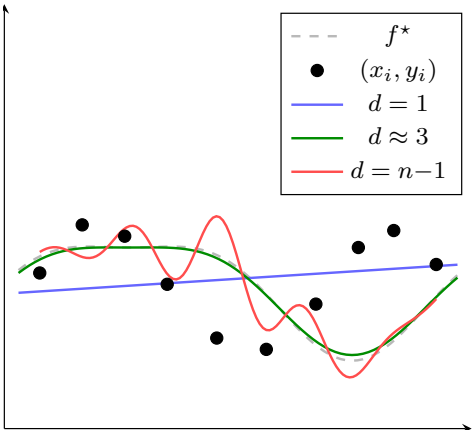
$$f_\theta(x) = \theta_0 + \theta_1 x + \theta_2 x^2 + \cdots + \theta_d x^d$$

La estructura está fija; buscamos **coeficientes**

Problema de aprendizaje

$$\min_{\theta \in \mathbb{R}^{d+1}} \mathcal{L}(\theta) \;=\; \frac{1}{n} \sum_{i=1}^{n} \big(f_\theta(x_i) - y_i\big)^2$$

- d pequeño $\Rightarrow$ clase rígida $\Rightarrow$ underfitting.
- $d = n - 1 \Rightarrow$ interpol. exacta $\Rightarrow$ overfitting.
- d intermedio $\Rightarrow$ balance precisión/generalización.



Mismo gráfico, nueva interpretación: cada curva corresponde a un $\theta_d^\star$ distinto.

¿Qué clase paramétrica usar?

Hasta ahora: polinomios de grado d.

Limitaciones de los polinomios:

- En altas dimensiones ($x \in \mathbb{R}^p$, p grande) el número de monomios crece **exponencialmente**.
- No hay garantía de que la clase sea lo suficientemente expresiva.

Idea clave

Construir una clase paramétrica **flexible y escalable** componiendo operaciones simples:

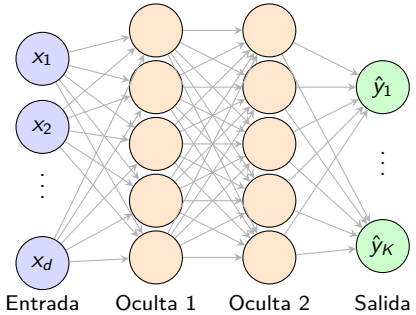
$$f_\theta = f^{(L)} \circ f^{(L-1)} \circ \cdots \circ f^{(1)}$$

donde cada *capa* $f^{(\ell)}$ aplica una transformación afín seguida de una no linealidad.

$\Rightarrow$ **Perceptrón Multicapa (MLP):** la clase paramétrica universal por excelencia.

(Teorema de aproximación universal: bajo condiciones suaves, un MLP de una capa oculta puede aproximar cualquier función continua con la precisión deseada.)

Perceptrón Multicapa (MLP)



Definición

Un MLP es una función $f(x;\theta)$ obtenida por composición de capas:

$$f = \sigma_L \circ A^{(L)} \circ \cdots \circ \sigma_1 \circ A^{(1)}$$

donde $A^{(\ell)}(h) = W^{(\ell)}h + b^{(\ell)}$ es una transformación afín y σ_ℓ es una función de activación no lineal (ReLU, tanh, ...).

- **Entrada:** $x \in \mathbb{R}^d$
- **Capas ocultas:** representaciones intermedias $h^{(\ell)}$
- **Salida:** $\hat{y} = f(x;\theta) \in \mathbb{R}^K$

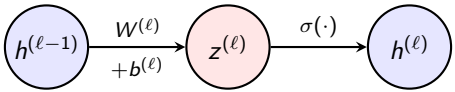
MLP como clase funcional

Definición

Un Perceptrón Multicapa (MLP) computa su salida mediante composiciones recursivas:

$$z^{(\ell)} = W^{(\ell)} h^{(\ell-1)} + b^{(\ell)}$$
$$h^{(\ell)} = \sigma(z^{(\ell)})$$

con $h^{(0)} = x$ y salida final $f(x;\theta) = z^{(L)}$.



- **Familia de funciones:** $\mathcal{F} = \{f(\cdot;\theta) : \theta \in \mathbb{R}^p\}$
- **Parámetros (θ):** Pesos $W^{(\ell)}$, sesgos $b^{(\ell)}$.
- **Variables:** $z^{(\ell)}$ (pre-act.), $h^{(\ell)}$ (activaciones).

Espacio de parámetros

Vector de Parámetros

Agrupa todos los pesos y sesgos de las capas de la red:

$$\theta = \left(\text{vec}(W^{(1)}), b^{(1)}, \ldots, \text{vec}(W^{(L)}), b^{(L)}\right) \in \mathbb{R}^p$$

- **Alta dimensionalidad:** La cantidad de parámetros p suele ser gigantesca (millones o billones), posibilitando la sobreparametrización ($p \gg n$).
- **Dependencia no lineal:** La función $f(x;\theta)$ depende de θ de forma fuertemente no lineal por las activaciones σ.
- **Simetrías funcionales:** Múltiples θ distintos pueden representar la misma función (e.g., al permutar neuronas en las capas ocultas).
- **Geometría compleja:** El paisaje de la función de pérdida es **no convexo**, repleto de mínimos locales y puntos silla.

Problema empírico

Minimización del Riesgo Empírico (ERM)

Dado un conjunto de entrenamiento $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$, buscamos los parámetros que minimizan el error empírico:

$$\min_{\theta \in \mathbb{R}^p} \hat{\mathcal{R}}(\theta) = \min_{\theta \in \mathbb{R}^p} \frac{1}{n} \sum_{i=1}^{n} \ell\big(f(x_i; \theta), y_i\big)$$

- **Función de pérdida (ℓ):** Cuantifica la penalidad entre la predicción del modelo $f(x_i; \theta)$ y la etiqueta verdadera y_i.
- **Meta real vs. práctica:** El objetivo real es minimizar el *riesgo esperado* $\mathbb{E}[\ell(f(x; \theta), y)]$, pero solo podemos minimizar su aproximación muestral $\hat{\mathcal{R}}(\theta)$.

Obstáculo algorítmico

A diferencia de los modelos lineales o convexos, debido a la estructura composicional de f, esta función objetivo $\hat{\mathcal{R}}(\theta)$ es fuertemente **no convexa**.

¿Por qué el entrenamiento no es convexo?

Visto en U3 · Condición de segundo orden

La red más chica posible: una entrada, una neurona lineal oculta, una salida, y un único dato $(x, y) = (1, 1)$:

$$f(x; a, b) = b\,a\,x, \qquad \mathcal{L}(a, b) = (ab - 1)^2.$$

¿Por qué el entrenamiento no es convexo?

La red más chica posible: una entrada, una neurona lineal oculta, una salida, y un único dato $(x, y) = (1, 1)$:

$$f(x; a, b) = b\,a\,x, \qquad \mathcal{L}(a, b) = (ab - 1)^2.$$

$$\nabla \mathcal{L} = 2(ab - 1)\begin{pmatrix} b \\ a \end{pmatrix}, \qquad \nabla^2 \mathcal{L}(0, 0) = \begin{pmatrix} 0 & -2 \\ -2 & 0 \end{pmatrix}.$$

- $(0, 0)$ es punto crítico, pero el hessiano tiene autovalores ± 2: es un **punto silla**.
- Por la U3, $\nabla^2 \mathcal{L} \not\succeq 0$ alcanza para concluir que $\mathcal{L}$ no es convexa.

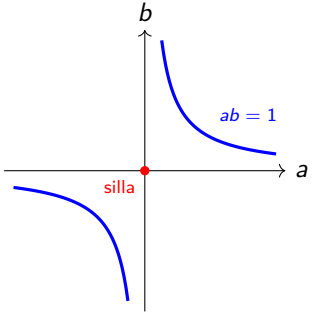
¿Por qué el entrenamiento no es convexo?

La red más chica posible: una entrada, una neurona lineal oculta, una salida, y un único dato $(x, y) = (1, 1)$:

$$f(x; a, b) = b\,a\,x, \qquad \mathcal{L}(a, b) = (ab - 1)^2.$$

$$\nabla \mathcal{L} = 2(ab - 1)\begin{pmatrix} b \\ a \end{pmatrix}, \qquad \nabla^2 \mathcal{L}(0,0) = \begin{pmatrix} 0 & -2 \\ -2 & 0 \end{pmatrix}.$$

- $(0, 0)$ es punto crítico, pero el hessiano tiene autovalores ± 2: es un **punto silla**.
- Por la U3, $\nabla^2 \mathcal{L} \not\succeq 0$ alcanza para concluir que $\mathcal{L}$ no es convexa.



El conjunto de minimizadores $\{ab = 1\}$ no es convexo ni conexo. En un problema convexo siempre es convexo (U3–U4). La simetría $(a, b) \mapsto (-a, -b)$ es un ejemplo de las simetrías del espacio de parámetros.

Optimización no convexa

Condiciones de optimalidad

Condición de Primer Orden (Puntos críticos)

Para que una configuración de parámetros θ^* sea un mínimo local, es condición necesaria que su gradiente se anule:

$$\nabla_\theta \hat{\mathcal{R}}(\theta^*) = 0$$

- **No suficiencia:** A diferencia de la optimización convexa, $\nabla_\theta \hat{\mathcal{R}}(\theta^*) = 0$ no garantiza hallar un mínimo global, ni siquiera local.
- **Diversidad de puntos críticos:** Esta condición se cumple tanto en mínimos locales, máximos locales, como en puntos silla.
- **Condición de Segundo Orden:** Para ser mínimo local, se requiere además que la curvatura sea no-negativa en toda dirección, es decir, que la matriz Hessiana cumpla $\nabla^2_\theta \hat{\mathcal{R}}(\theta^*) \succeq 0$.

Estructura de puntos críticos

El paisaje de optimización en redes neuronales presenta distintos tipos de puntos donde $\nabla_\theta \hat{\mathcal{R}}(\theta) \approx 0$:

- **Mínimos locales:** Puntos donde toda dirección incrementa la pérdida. En redes sobreparametrizadas, la literatura sugiere que la mayoría tiene un valor de pérdida muy cercano al óptimo global.
- **Puntos silla (Saddle points):** Puntos que son un mínimo en algunas direcciones y un máximo en otras (la matriz Hessiana posee autovalores tanto positivos como negativos).
- **Regiones planas (Plateaus):** Extensas zonas con gradiente casi nulo que pueden estancar algoritmos de primer orden muy tempranamente.

Hecho clave en alta dimensión ($p \gg 1$)

La probabilidad de que las p direcciones de curvatura sean todas positivas (condición de mínimo local) decae exponencialmente con p. Por ello, **predominan abrumadoramente los puntos silla** por sobre los mínimos locales subóptimos con alta pérdida.

Sobreparametrización

Régimen de alta capacidad ($p \gg n$)

Una red neuronal moderna típicamente posee muchísimos más parámetros p que ejemplos de entrenamiento n.

Theorem (Interpolación perfecta)

En este régimen, usualmente existen configuraciones de parámetros θ que logran pérdida casi nula en el conjunto de entrenamiento (memorización):

$$f(x_i; \theta) = y_i \quad \forall i = 1, \ldots, n$$

- **Contradice la intuición clásica:** Tradicionalmente, un modelo que interpola los datos ruidosos debería sufrir un *overfitting* catastrófico.
- **Generalización exitosa:** En la práctica, redes sobreparametrizadas que interpolan el training set suelen lograr un desempeño excelente (buena generalización).

Dilema Bias-Variance en Deep Learning

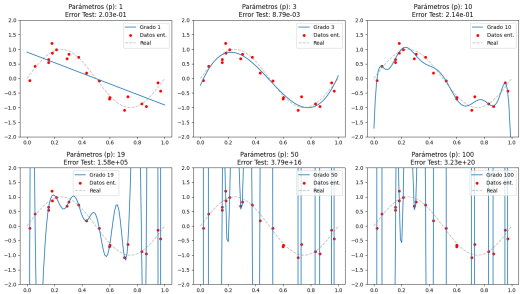
- **Curva Clásica en U:** Modelos con poca capacidad sufren de alto sesgo (underfitting). Al subir la capacidad baja el sesgo, pero luego aumenta bruscamente la varianza (overfitting clásico).
- **Doble Descenso (Double Descent):** Descubrimiento moderno. Al incrementar la capacidad aún más (régimen de interpolación), la curva de test vuelve a descender. Modelos *inmensos* generalizan mejor.
- **Regularización implícita:** La elección del optimizador (e.g., descenso por gradiente) dicta qué solución se encuentra. SGD posee un sesgo inductivo que prefiere de manera natural soluciones interpolantes de "baja complejidad".

La generalización no depende solo de la capacidad del modelo, sino también del sesgo inducido por el algoritmo de entrenamiento.

Curva en U

Tradicionalmente, compromiso entre **Sesgo** y **Varianza**.

- **Baja Capacidad:** Alto sesgo (Underfitting).
- **Capacidad Óptima:** punto ideal (estadística).
- **Alta Capacidad:** Alta varianza (Overfitting catastrófico).



Intuición Clásica

Minimizar el riesgo empírico $R_{emp} \to 0$ mediante interpolación pura era considerado un error de diseño.

Doble Descenso

Actualmente se comprobó que el error de test vuelve a descender tras el umbral de interpolación.

Umbral de Interpolación ($p \approx n$): El pico de varianza donde el modelo memoriza el ruido con inestabilidad numérica.

Régimen Sobreparametrizado ($p \gg n$): Modelos inmensos que logran una generalización superior.

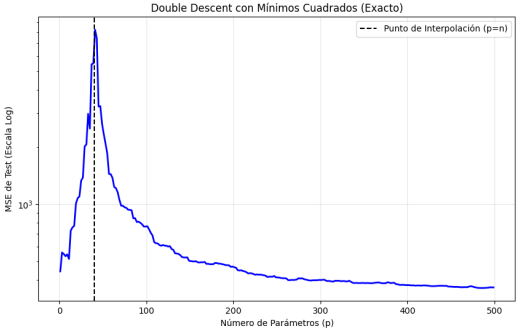
Matemáticamente: $f(x_i; \theta) = y_i \quad \forall i = 1, \ldots, n$

Regularización implícita

¿Por qué generalizan los modelos sobreparametrizados? La clave no es solo el modelo, sino el **algoritmo**.

- **Sesgo Inductivo:** Algoritmos como SGD o Adam prefieren de forma natural soluciones de "baja complejidad".
- **Mínimos Cuadrados:** En el régimen $p > n$, la pseudoinversa de Moore-Penrose selecciona la solución de **norma mínima** (el ejemplo que resolvimos con dualidad en la U9):

$$\min_{\theta} \|\theta\|_2 \quad \text{sujeto a} \quad X\theta = y$$



Conclusión

La optimización actúa como un regularizador implícito que "suaviza"la superficie de error.

Regularización explícita

Objetivo

Controlar la complejidad efectiva del modelo y evitar que dependa rígidamente de perturbaciones o ruido en los datos.

- **Weight decay (L2):** Penaliza parámetros de gran magnitud añadiendo $\lambda\|\theta\|_2^2$ a la función de pérdida. Es la regularización Ridge de la clase 1.
- **Dropout:** Omite aleatoriamente activaciones de neuronas durante el entrenamiento. Obliga a la red a desarrollar representaciones redundantes y robustas.
- **Early stopping:** Interrumpe el entrenamiento (antes de llegar al mínimo empirico) justo cuando el error en validación comienza a deteriorarse.
- **Data Augmentation:** Aumentar artificialmente el tamaño de $\mathcal{D}$ aplicando transformaciones invariantes (rotaciones, ruidos).

Funciones de pérdida

ANN para clasificación

Función de Pérdida (Loss Function)

Mide la discrepancia entre la distribución real y (one-hot) y la predicha $\hat{p}$:

$$L(y, \hat{p}) = -\sum_{k=1}^{K} y_k \log(\hat{p}_k)$$

Capa Softmax

La última capa debe dar esas probabilidades $\hat{p}$:

$$\hat{p}_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}$$

El gradiente respecto a los logits z es la diferencia entre la probabilidad predicha y la verdadera:

$$\frac{\partial L}{\partial z_k} = \hat{p}_k - y_k$$

Softmax

Definición

Dado un vector de *logits* $z \in \mathbb{R}^K$, la función Softmax asigna a cada clase k una probabilidad:

$$p_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}$$

- **Mapeo a probabilidades:** Transforma salidas abstractas reales en una distribución de probabilidad válida ($p \geq 0, \sum_k p_k = 1$).
- **Invarianza por traslación:** $\text{softmax}(z) = \text{softmax}(z + c\mathbf{1})$ para constante c, propiedad útil analítica y computacionalmente.
- **Amplificación:** La exponencial favorece al logit máximo, tendiendo a asignar probabilidades fuertemente dominantes a la clase ganadora.

Cross-Entropy (Entropía Cruzada)

Definición empírica

Para una observación con distribución predicha p y etiqueta verdadera categórica o *one-hot* y, definimos:

$$\ell(p, y) = -\sum_{k=1}^{K} y_k \log p_k$$

Theorem

Minimizar la Cross-Entropy empírica equivale analíticamente a:

- ***KL Divergence:*** *Minimizar la divergencia de Kullback-Leibler* $\mathrm{KL}(y \| p)$*, es decir, alinear la distribución predicha p a la verdadera y.*
- ***MLE:*** *Maximizar la verosimilitud (Maximum Likelihood Estimation) de los datos de entrenamiento bajo un modelo probabilístico categórico.*

Estabilidad numérica y Log-Sum-Exp

Problema: Overflow y Underflow

Los exponentes computados en Softmax, e^{z_k}, pueden ser inmensos si $z_k \gg 0$, retornando `NaN` por desbordamiento (Overflow). Si $z_k \ll 0$, evalúan a cero (Underflow), creando inestabilidades como $\log(0)$ al evaluar la pérdida.

Solución: Traslación y Log-Sum-Exp (LSE)

Definiendo $m = \max_j z_j$ y empleando la invarianza por traslación:

$$\log \sum_j e^{z_j} = m + \log \sum_j e^{z_j - m}$$

Esta maniobra asegura que el mayor exponente evaluado sea precisamente 0 (ya que $z_j - m \leq 0$), previniendo el overflow y calculándose de forma estable.

Además, $z \mapsto \log \sum_j e^{z_j}$ es **convexa** (ejemplo de la U3), así que la entropía cruzada es convexa en los logits z: la

Gradiente Combinado (Softmax + CE)

Cuando se calcula con la regla de la cadena la derivada conjunta de la de pérdida de Entropía Cruzada evaluada sobre las probabilidades del Softmax, se produce una enorme cancelación de términos.

Resultado clave

La derivada de la pérdida ℓ respecto a la pre-activación de salida z_k es:

$$\frac{\partial \ell}{\partial z_k} = p_k - y_k$$

- **Forma simple:** La magnitud del gradiente es directamente proporcional al error de predicción.
- **Estabilidad y Eficiencia:** En la práctica (como en PyTorch `CrossEntropyLoss`), Softmax y Entropía Cruzada se computan internamente acopladas para evitar inestabilidades de evaluar probabilidades y luego su logaritmo por separado.

Backpropagation

Forward Pass

- **Propagación hacia adelante:** Se computan secuencialmente las pre-activaciones $z^{(\ell)}$ y activaciones $h^{(\ell)}$ capa por capa.
- **Cálculo de la pérdida:** Finaliza evaluando $\ell(f(x_i; \theta), y_i)$.
- **Requisito de Memoria:** Se almacenan las activaciones intermedias (grafo computacional) ya que se requerirán posteriormente para el cálculo de los gradientes.

Backward Pass (Backprop)

Propagación del error

Recomputamos el error del output $\delta^{(\ell)} = \frac{\partial L}{\partial z^{(\ell)}}$ propagando su impacto desde la salida hacia la entrada:

$$\delta^{(\ell)} = (W^{(\ell+1)})^T \delta^{(\ell+1)} \odot \sigma'(z^{(\ell)})$$

- **Derivadas de pesos:** Sabiendo δ, el gradiente respecto a los parámetros es simplemente $\frac{\partial L}{\partial W^{(\ell)}} = \delta^{(\ell)} (h^{(\ell-1)})^T$.
- **Sistema en reversa:** Aplica la regla de la cadena reutilizando resultados para mayor eficiencia.

Regla de la Cadena como Grafo Computacional

Estructura Composicional

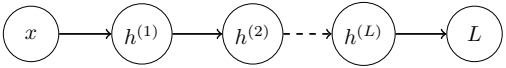
$$\frac{\partial L}{\partial \theta} = \prod \frac{\partial}{\partial (\cdot)}$$

Interpretación Algorítmica

Backpropagation es un algoritmo de **programación dinámica** sobre el grafo computacional de la red. Evita recomputar multiplicaciones matriciales anidadas (derivadas parciales) memorizando y propagando el término $\delta^{(\ell+1)}$.

Backpropagation como programación dinámica

Grafo computacional (forward)

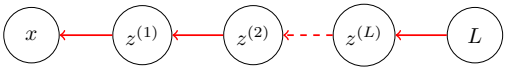


Estructura composicional:

$$L = f^{(L)} \circ f^{(L-1)} \circ \cdots \circ f^{(1)}(x)$$

$$z^{(\ell)} = W^{(\ell)} h^{(\ell-1)} + b^{(\ell)},$$
$$h^{(\ell)} = \sigma(z^{(\ell)})$$

Backward (gradientes)



$$\delta^{(\ell)} = \frac{\partial L}{\partial z^{(\ell)}}$$

$$\delta^{(\ell)} = (W^{(\ell+1)})^\top \delta^{(\ell+1)} \odot \sigma'(z^{(\ell)})$$

Backpropagation aplica la regla de la cadena de forma eficiente reutilizando derivadas intermedias.

- Sin reutilización: costo exponencial
- Con backprop: costo lineal en la profundidad

Costo Computacional: Training vs. Inferencia

La diferencia radical entre entrenar y predecir radica en **guardar resultados**.

Entrenamiento (Backprop)

- **Cómputo:** $\approx 3\times$ FLOPs (Forward + 2 Gradientes).
- **VRAM:** Crítica. Requiere almacenar las **activaciones** de cada capa para aplicar la regla de la cadena.
- **Estado:** Dinámico y pesado.

Inferencia (Forward)

- **Cómputo:** $1\times$ FLOPs. Operación puramente lineal/no-lineal.
- **VRAM:** Mínima. Solo requiere cargar el vector de parámetros θ.
- **Latencia:** Óptima para despliegue en tiempo real.

El tamaño del batch durante el entrenamiento escala linealmente el uso de memoria debido a las activaciones guardadas.

CNN

De MLP a CNN: motivación

Problema con MLP en imágenes

Una capa fully-connected conecta **cada** neurona de salida con **todas** las entradas:

$$h_j = \sigma\left(\sum_{i=1}^{H\cdot W\cdot C} W_{ji}\, x_i + b_j\right)$$

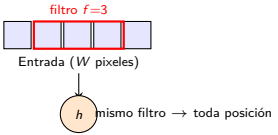
Para una imagen $224 \times 224 \times 3$ y 1024 neuronas: $\approx$ **150 M** parámetros *solo en esa capa*.

Dos problemas concretos:

- **Explosión paramétrica:** ineficiente, sobreajuste.
- **Sin estructura espacial:** el modelo no sabe que los píxeles vecinos están relacionados.

Solución CNN: dos restricciones

1. **Conectividad local:** cada neurona solo ve un vecindario pequeño (*receptive field* $f \times f$).
2. **Pesos compartidos:** el mismo filtro se aplica en todas las posiciones.



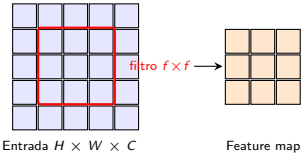
Capas Convolucionales (CNN)

Definición discreta

La pre-activación del mapa de características k en posición (i, j):

$$y_{i,j,k} = \sum_{c,u,v} K_{u,v,c,k}\, x_{i+u,\, j+v,\, c}$$

$K \in \mathbb{R}^{f \times f \times C \times K}$ es el tensor de filtros; (u, v) recorre el receptive field.



Comparación de parámetros:

Capa	Parámetros
FC ($224^2 \times 3 \to 1024$)	$\sim$150 M
Conv (f=3, C=3, K=64)	$\sim$1.7 K

- **Localidad:** solo se accede a un parche $f \times f$ de la entrada.
- **Receptive field:** el área de entrada que influye en una salida crece capa a capa.
- **Reducción de parámetros:** $f^2 CK \ll (HWC)(H'W'K)$ del MLP equivalente.

Parameter Sharing e Interpretación Matricial

- **Matrices tipo Toeplitz:** La convolución puede expresarse internamente como un producto matriz-vector tradicional reordenando la entrada en bloque con una matriz rala circulante.
- **Parameter Sharing:** El mismo detector de patrones (filtro) comparte sus pesos para toda la imágen completa.
- **Invarianza local:** Genera una *equivariancia por traslación* y causa una impactante reducción de parámetros (θ) respecto de la capa conectada clásica (MLP).

Control de resolución y submuestreo espaciales

Para escalar CNNs, gestionamos las resoluciones de los mapas de entrada (Feature Maps):

- **Stride (Paso):** Salto periódico del filtro sobre el píxel de entrada. Si > 1 actúa reduciendo bruscamente la resolución, economizando costo computacional y FLOPs.
- **Padding:** Bordes falsos (usualmente ceros) incorporados para evitar el achicamiento natural de las esquinas o garantizar divisibilidades.
- **Pooling (Submuestreo):** Funciones sin parámetros locales y estrucuturales (como `MaxPool`) que dan invarianza local.

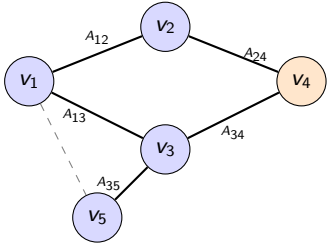
GNN

De CNN a GNN: vecinos definidos por similaridad

Limitación de CNN: la noción de "vecino" está fija en una grilla regular ($f \times f$ píxeles).

Muchos dominios reales **no tienen grilla**:

- Moléculas (átomos = nodos, enlaces = aristas)
- Redes sociales (usuarios = nodos)
- Grafos de conocimiento, meshes 3D, ...



A_{ij}: similitud entre v_i y v_j

Idea central de GNN

Reemplazar la grilla fija por una **matriz de similaridad/adyacencia** $A \in \mathbb{R}^{n \times n}$ que codifica qué nodos son vecinos y con qué peso.

Es la misma matriz A (y su laplaciano $L = D - A$) del clustering espectral de la clase 2.

Capa de agregación en GNN

Graph Convolution (GCN — Kipf & Welling, 2017)

La actualización de la representación del nodo i en la capa ℓ:

$$h_i^{(\ell)} = \sigma\left(W^{(\ell)} \sum_{j \in \mathcal{N}(i)} \frac{A_{ij}}{\sqrt{d_i d_j}} h_j^{(\ell-1)} \right)$$

donde $\mathcal{N}(i)$ son los vecinos de i (incluye i) y $d_i = \sum_j A_{ij}$ es el grado.

$$H^{(\ell)} = \sigma\left(\tilde{A}\, H^{(\ell-1)}\, W^{(\ell)} \right)$$

$\tilde{A} = D^{-1/2}(A + I)D^{-1/2}$ m. de adyacencia normalizada.

Analogía con CNN

CNN	GNN
Grilla $H \times W$	Grafo $G = (V, E)$
Píxeles vecinos	$\mathcal{N}(i)$ por A
Filtro fijo K	Pesos $W^{(\ell)}$ compartidos
Feature map	$H^{(\ell)} \in \mathbb{R}^{n \times d}$

- **Parameter sharing:** $W^{(\ell)}$ es el mismo para todos los nodos.
- **Permutation equivariance:** reordenar nodos no cambia el resultado (si A se reordena igual).

Unificación: MLP $\rightarrow$ CNN $\rightarrow$ GNN



MLP
$h = \sigma(Wx + b)$
$W \in \mathbb{R}^{d' \times d}$ densa
sin estructura geométrica

CNN
W dispersa (rala), Toeplitz
vecinos = grilla fija
equivariante a traslación

GNN
vecinos = $\mathcal{N}(i)$ por A
A puede ser aprendida
equivariante a permutación

CNN y GNN son **restricciones estructuradas** del MLP: imponen un prior sobre qué entradas interactúan, reduciendo parámetros y aprovechando la geometría del dominio.

Complejidad

Análisis General de Complejidad

Al escalar o desplegar arquitecturas en producción (por ej. GPUs limitadas), los ingenieros monitorean:

- **Parámetros vs FLOPs:** En arquitecturas canónicas de visión, las capas Fully Connected acaparan inmensamente los parámetros disponibles de memoria base, mientras que las Convoluciones monopolizan el volumen y operaciones aritméticas de cómputo (FLOPs).
- **Memoria en Entrenamiento:** Limitada críticamente por los estados temporales de Activación, dependientes fuertemente del Batch size y resolución, requiriendo usar Memory Tracking a lo largo del paso Backprop.
- **Latencia (Inferencia):** Al hacer deployment, requiere balancear el ancho de red con los canales paralelos, tradeando Precisión en generalización vs Costo computacional final en tiempo real.

Redes KAN

Redes KAN: idea central

- **KAN** = *Kolmogorov-Arnold Networks*: arquitectura inspirada en el teorema de representación de Kolmogorov-Arnold.
- En lugar de aprender solo pesos escalares, KAN aprende **funciones univariadas** en las conexiones.
- Intuición: una capa KAN reemplaza operaciones lineales rígidas por combinaciones de funciones más expresivas.
- Esto puede mejorar la **interpretabilidad**: cada arista tiene una curva aprendida que puede visualizarse.

KAN vs MLP: forma matemática

MLP clásico

$$h_j = \sigma\left(\sum_i w_{ji} x_i + b_j\right)$$

- Parámetros: pesos w_{ji} y sesgos.
- No linealidad fija en nodos.

Capa KAN

$$h_j = \sum_i \phi_{ji}(x_i)$$

- Cada ϕ_{ji} es una función aprendida.
- No linealidad distribuida en aristas.

Lectura conceptual

El MLP aprende *coeficientes*; KAN aprende *curvas*. Esto amplía la flexibilidad funcional por conexión.

Cómo se parametrizan las funciones en KAN

- En implementaciones prácticas, cada ϕ_{ji} suele representarse con **splines** (p. ej. B-splines).
- Una forma común:

$$\phi_{ji}(x) = w_{ji}\, x + \sum_{m=1}^{M} c_{ji,m}\, B_m(x)$$

  donde $B_m(x)$ son bases spline y $c_{ji,m}$ coeficientes aprendibles.
- Esto combina una parte lineal + una corrección no lineal suave.
- Regularizaciones sobre los coeficientes pueden inducir funciones más simples e interpretables.

Ventajas y desafíos de KAN

Ventajas potenciales

- Mayor interpretabilidad (curvas por conexión).
- Buena aproximación con menos ancho en algunos problemas.
- Útil cuando interesa extraer relaciones funcionales explícitas.

Desafíos prácticos

- Entrenamiento más costoso que MLP en muchos casos.
- Más hiperparámetros (número de nodos de spline, regularización).
- Escalabilidad en tareas muy grandes aún en estudio.

Mensaje final: KAN es una alternativa prometedora a MLP cuando priorizamos interpretabilidad y estructura funcional aprendida.

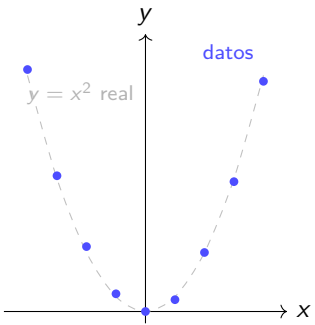
Ejemplo 1D: dataset y objetivo

Datos de entrenamiento

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{9}, \quad x_i \in [-2, 2], \quad y_i = x_i^2 + \varepsilon_i$$

con ruido pequeño ε_i.

x_i	-2	-1.5	-1	-0.5	0	0.5	1	1.5	2
y_i	4.1	2.3	1.1	0.3	0.0	0.2	1.0	2.2	3.9



Problema

Aprender $\hat{f}$ minimizando error cuadrático medio:

$$\min_{\theta} \frac{1}{n} \sum_{i=1}^{n} \left(\hat{f}_\theta(x_i) - y_i\right)^2$$

Paso 1: ajuste con MLP

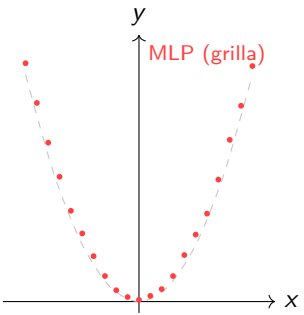
Modelo

$$\hat{f}_{\text{MLP}}(x) = W_2\,\sigma(W_1 x + b_1) + b_2$$

con una capa oculta chica (ej.: 8 neuronas, activación ReLU/tanh).

Entrenamiento

- Inicialización aleatoria de pesos.
- Descenso por gradiente sobre MSE.
- Predicción en una grilla densa de x para visualizar.



Lectura

El MLP aproxima bien, pero la curva aprendida aparece como **muestra discreta** (nube de puntos) al evaluarlo en la grilla.

Paso 2: ajuste con KAN

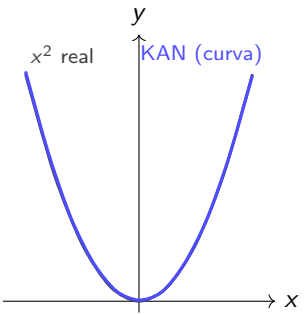
Modelo

$$\hat{f}_{\text{KAN}}(x) = \sum_{j=1}^{m} \phi_j(x)$$

donde cada ϕ_j es una función univariada parametrizada (ej.: spline).

Entrenamiento

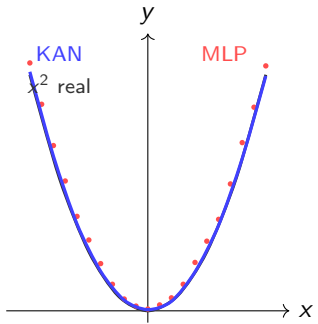
- Se ajustan coeficientes de las bases de cada ϕ_j.
- Regularización para evitar curvas demasiado oscilantes.
- Resultado: función suave y legible por conexión.



Lectura

En KAN, la hipótesis aprendida es directamente una **curva explícita** $\phi(x)$, más interpretable en 1D.

Comparación final: MLP (nube) vs KAN (curva)



Qué mostrar en clase

- Ambos aproximan la misma relación no lineal.
- **MLP:** salida visualizada como evaluaciones puntuales.
- **KAN:** salida visualizada como curva funcional explícita.
- El contraste ayuda a explicar **interpretabilidad**.

Takeaway

En 1D, KAN vuelve tangible la idea de "aprender funciones" y no solo pesos.

Backpropagation: MLP vs KAN

MLP (muy alineado a backprop)

$$h_j = \sigma\left(\sum_i w_{ji} x_i + b_j\right)$$

- Operaciones estándar: producto matricial + activación.
- Jacobianos simples y altamente vectorizables.
- Implementación muy optimizada en GPUs/TPUs.

$$\frac{\partial \mathcal{L}}{\partial w_{ji}} = \frac{\partial \mathcal{L}}{\partial h_j}\, \sigma'(a_j)\, x_i$$

KAN (también permite backprop)

$$h_j = \sum_i \phi_{ji}(x_i), \quad \phi_{ji}(x) = \sum_m c_{ji,m} B_m(x)$$

- Sigue siendo diferenciable si ϕ y sus bases lo son.
- Backprop calcula gradientes sobre coeficientes $c_{ji,m}$.
- Más costo por evaluar/derivar funciones por arista.

$$\frac{\partial \mathcal{L}}{\partial c_{ji,m}} = \frac{\partial \mathcal{L}}{\partial h_j}\, B_m(x_i)$$

Respuesta corta

Cierre

Resumen General

- **Problema central:** El entrenamiento de DL se abstrae como la **optimización de funciones no convexas** en altísimas dimensiones bajo Regímenes Empíricos (ERM).
- **Formulación analítica:** Con Softmax y Entropía Cruzada, aproximamos de manera estable los problemas numéricos interpretando la red matemáticamente como MLE.
- **Algoritmo central:** Usamos sistemáticamente Backpropagation, una versión robusta y vectorizada de rule chaining, para entrenar y recomputar.
- **Tipología de clases funcionales:** Las variaciones estructurales (arquitecturas, prior inductive biases como CNNs) permiten inyectar conocimiento y atacar distintos tipos de eficiencias computacionales.

Conexión con la materia

Las condiciones de optimalidad (U3–U4) siguen valiendo, pero sólo como condiciones locales; lo que se pierde es la equivalencia local = global. En la próxima clase volvemos a los métodos de descenso de la U4 y vemos sus versiones estocásticas, que son las que se usan para entrenar redes.

Bibliografía

- Goodfellow et al., Deep Learning
- CS231n (Stanford)
- Zhang et al. (generalización)
- He et al. (ResNet)
- Ioffe & Szegedy (BatchNorm)

Apéndice: Matriz de Toeplitz y convolución

Definición

Una matriz $T \in \mathbb{R}^{m \times n}$ es de **Toeplitz** si cada diagonal descendente es constante:

$$T_{ij} = t_{i-j}.$$

Conexión con CNN

La convolución 1D de una entrada $x \in \mathbb{R}^n$ con un filtro k de tamaño f es equivalente a:

$$y = T\,x, \quad T \in \mathbb{R}^{(n+f-1)\times n}$$

$$T = \begin{pmatrix} k_0 & & & \\ k_1 & k_0 & & \\ k_2 & k_1 & \ddots & \\ & k_2 & \ddots & k_0 \\ & & \ddots & k_1 \\ & & & k_2 \end{pmatrix}$$

- **Parámetros compartidos:** toda la fila de T comparte los mismos f valores del filtro.
- **Esparsidad:** T tiene $\mathcal{O}(f)$ valores distintos frente a $\mathcal{O}(mn)$ de una FC densa.
- **Eficiencia:** en 2D, la estructura circulante permite implementar la convolución vía FFT en $\mathcal{O}(N \log N)$ en lugar de $\mathcal{O}(N^2)$.
- **Equivariancia:** si la entrada se desplaza, la salida se desplaza igual.

Apéndice: múltiples filtros por capa

En la práctica cada capa convolucional aplica K **filtros distintos** simultáneamente, generando K mapas de características.

Un filtro $\rightarrow$ una Toeplitz

Filtro k_j de tamaño f, entrada 1D $x \in \mathbb{R}^n$:

$$y_j = T_j\, x, \quad T_j \in \mathbb{R}^{n' \times n}$$

T_j es Toeplitz con los valores de k_j en las diagonales.

Con K filtros, apilamos:

$$\begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_K \end{pmatrix} = \underbrace{\begin{pmatrix} T_1 \\ T_2 \\ \vdots \\ T_K \end{pmatrix}}_{T^{(\ell)} \in \mathbb{R}^{(Kn') \times n}} x$$

Analogía con MLP

	MLP	**CNN**
Matriz de capa	$W^{(\ell)}$ densa	$T^{(\ell)}$ bloques Toeplitz
Filas	d_ℓ	$K \cdot n'$
Columnas	$d_{\ell-1}$	$n \cdot C$
Parám. libres	$d_\ell\, d_{\ell-1}$	$K \cdot f \cdot C$

Mensaje: la CNN es un MLP cuya matriz de pesos está *restringida* a tener estructura Toeplitz en bloques. Cada bloque = un filtro; parámetros libres $K \cdot f \cdot C \ll d_\ell\, d_{\ell-1}$.

Apéndice: Derivada de la Función Softmax

La derivada de la probabilidad p_i respecto al logit z_j se usa en *backpropagation*. Sea $p_i = \frac{e^{z_i}}{\sum_k e^{z_k}}$. Definimos $S = \sum_k e^{z_k}$. La derivada $\frac{\partial p_i}{\partial z_j}$ presenta dos casos:

1. **Caso Directo ($i = j$):**

$$\frac{\partial p_i}{\partial z_i} = \frac{e^{z_i} S - e^{z_i} e^{z_i}}{S^2} = \frac{e^{z_i}}{S}\left(1 - \frac{e^{z_i}}{S}\right) = \mathbf{p_i(1 - p_i)}$$

2. **Caso Cruzado ($i \neq j$):**

$$\frac{\partial p_i}{\partial z_j} = \frac{0 \cdot S - e^{z_i} e^{z_j}}{S^2} = -\frac{e^{z_i}}{S}\frac{e^{z_j}}{S} = -\mathbf{p_i p_j}$$

Usando delta de Kronecker

$$\frac{\partial p_i}{\partial z_j} = p_i(\delta_{ij} - p_j) \quad \text{donde } \delta_{ij} = \begin{cases} 1 & \text{si } i = j \\ 0 & \text{si } i \neq j \end{cases}$$